TÀI LIỆU ĐẶC TẢ CHI TIẾT TOÀN BỘ API VÀ LOGIC NGHIỆP VỤ BACKEND — SPRINT 2

- **Dự án:** OISMS — Omnichannel Inventory & Sales Management System
- **Học phần:** Đồ án Chuyên ngành / Kỹ nghệ Phần mềm (Trường Đại học Giao thông Vận tải TP.HCM - UTH)
- **Tác giả & Kiến trúc sư Backend 1 Lead:** Lê Võ Đăng Khoa (Trưởng nhóm / DevOps)
- **Đồng tác giả & Phụ trách Backend 2 Lead: Nhựt Phùng Minh** (Auth, Catalog, Product, Barcode & POS API)
- **Công nghệ nền tảng:** C# .NET 8 (ASP.NET Core Web API), Entity Framework Core 8, PostgreSQL 16+, Redis 7, Docker, Nginx
- **Quy mô Backend Sprint 2:** 10 Controllers, 46 Endpoints API, 194 Tests đạt chuẩn 100%, Code Coverage 94.83%

📊 BẢNG PHÂN ĐỊNH TRÁCH NHIỆM KỸ THUẬT (RACI MATRIX):

Phân định rõ ràng trách nhiệm thiết kế, triển khai mã nguồn C# và viết bài kiểm thử giữa hai kỹ sư Backend:

Phân hệ / Nhóm API	Quy mô	Thành viên phụ trách	Đóng góp kỹ thuật cụ thể
1. Xác thực & Phân quyền (`AuthController`)	5 Endpoints	Nhựt Phùng Minh *Backend 2 Lead*	Đăng ký Tenant, Login JWT, Token Rotation (Access 60p, Refresh 7d), Logout thu hồi token, API `/me`, băm mật khẩu BCrypt.
2. Cây danh mục & Thương hiệu (`CategoriesController`, `BrandsController`)	8 Endpoints	Nhựt Phùng Minh *Backend 2 Lead*	Cây danh mục nhiều tầng, **Thuật toán Chống vòng lặp đệ quy (Cycle Detection)** bằng PostgreSQL Advisory Lock, CRUD Brand.
3. Sản phẩm, SKU & Barcode (`ProductsController`)	10 Endpoints	Nhựt Phùng Minh *Backend 2 Lead*	CRUD Sản phẩm/SKU, Giá sỉ/lẻ, **Thuật toán EAN-13 Modulo 10**, **Thuộc tính động JSONB & Canonical Signature**.
4. Tra cứu Bán hàng POS (`PosCatalogController`)	2 Endpoints	Nhựt Phùng Minh *Backend 2 Lead*	Tìm kiếm siêu tốc và quét Barcode cho quầy POS (<50ms), tính tồn khả dụng Available theo đúng chi nhánh thu ngân.
5. Nhà cung cấp (`SuppliersController`)	3 Endpoints	Nhựt Phùng Minh *Backend 2 Lead*	Quản lý NCC, kiểm tra trùng MST / SĐT trong cùng Tenant.
6. Quản lý Chi nhánh & Kho hàng (`BranchController`, `WarehousesController`)	10 Endpoints	Khoa & Nhựt *Phối hợp*	Khoa xây dựng API kho/chi nhánh; Nhựt bổ sung logic nghiệp vụ **BE2-08** (chặn đổi branchId khi kho đã phát sinh số dư/giao dịch).
7. Nhập kho, Sổ cái & WAC (`GoodsReceiptsController`, `InventoryController`)	8 Endpoints	Lê Võ Đăng Khoa *Backend 1 Lead*	Thuật toán tính giá vốn bình quân WAC, Trigger PostgreSQL bảo vệ Sổ kho Append-Only bất biến, Engine Giữ chỗ tồn kho (Reservation).
8. Kiến trúc lõi & Multi-tenant (Clean Architecture, TenantGuard)	Hạ tầng	Lê Võ Đăng Khoa *Backend 1 Lead*	Clean Architecture 4 tầng, Global Query Filter, TenantGuard, OpenPrivilegedScope, Docker Compose & CI/CD.

👉 *Khối lượng thực hiện của **Nhựt Phùng Minh (Backend 2): 6 Controllers, 28/46 Endpoints API (60.9%), 3 Thuật toán lõi**.*

MỤC LỤC NỘI DUNG

1. **KIẾN TRÚC TỔNG THỂ & CÁC NGUYÊN TẮC THIẾT KẾ**
2. **CƠ CHẾ CÔ LẬP ĐA NGƯỜI THUÊ (MULTI-TENANT ENGINE)**
3. **ĐẶC TẢ CHI TIẾT 10 CONTROLLERS & 46 ENDPOINTS API**
   - 3.1. AuthController (5 Endpoints) Nhựt
   - 3.2. BranchController (5 Endpoints) Khoa
   - 3.3. WarehousesController (5 Endpoints) Khoa & Nhựt
   - 3.4. SuppliersController (3 Endpoints) Nhựt
   - 3.5. CategoriesController (4 Endpoints) Nhựt
   - 3.6. BrandsController (4 Endpoints) Nhựt
   - 3.7. ProductsController (10 Endpoints) Nhựt
   - 3.8. GoodsReceiptsController (4 Endpoints) Khoa
   - 3.9. InventoryController (4 Endpoints) Khoa
   - 3.10. PosCatalogController (2 Endpoints) Nhựt
4. **CÁC THUẬT TOÁN & LOGIC NGHIỆP VỤ CỐT LÕI**
   - 4.1. Duyệt nhập kho & Tính WAC trong DB Transaction Khoa
   - 4.2. Sổ cái kho bất biến (Append-Only Inventory Ledger) Khoa
   - 4.3. Giữ chỗ tồn kho & PostgreSQL Advisory Lock Khoa
   - 4.4. Chống vòng lặp Cây danh mục (Cycle Detection) Nhựt
   - 4.5. Sinh mã vạch chuẩn quốc tế EAN-13 (Modulo 10) Nhựt
   - 4.6. Thuộc tính động JSONB & Canonical Signature Nhựt
5. **MA TRẬN PHÂN QUYỀN TRUY CẬP (RBAC MATRIX)** Nhựt
6. **PHỤ LỤC: ĐỐI CHIẾU NHIỆM VỤ JIRA (OISMS-16, 27, 31)** Nhựt

1. KIẾN TRÚC TỔNG THỂ & CÁC NGUYÊN TẮC THIẾT KẾ Lê Võ Đăng Khoa phụ trách

Backend OISMS được xây dựng theo mô hình **Clean Architecture (Onion Architecture)** phân tách thành 4 tầng ranh giới nghiêm ngặt:

1. **OISM.Domain:** Chứa thực thể cốt lõi (`Entities`), sự kiện miền (`Domain Events`), enum trạng thái và các quy tắc nghiệp vụ bất biến. Tuyệt đối không phụ thuộc vào bất kỳ thư viện hay cơ sở dữ liệu nào bên ngoài.
2. **OISM.Application:** Chứa logic ứng dụng, triển khai mô hình CQRS / Feature-based (`Features/`), DTOs, hợp đồng dịch vụ (`Interfaces`), bộ kiểm tra tính hợp lệ (`Validators` - FluentValidation) và các thuật toán tính toán (WAC, Barcode, SkuAttribute).
3. **OISM.Infrastructure:** Chứa tầng truy cập dữ liệu EF Core (`AppDbContext`), cấu hình Fluent API (`Configurations/`), triển khai các dịch vụ kỹ thuật (`JwtTokenService` , `CurrentTenantService`), quản lý kết nối CSDL và migration.
4. **OISM.WebApi:** Tầng giao tiếp bên ngoài (Presentation Layer) chứa các `Controllers` , bộ lọc middleware, cấu hình Swagger OpenAPI và Dependency Injection.

2. CƠ CHẾ CÔ LẬP ĐA NGƯỜI THUÊ (MULTI-TENANT ENGINE)

Lê Võ Đăng Khoa phụ trách

Hệ thống hoạt động theo mô hình **Shared Database, Shared Schema**, nơi tất cả các cửa hàng (Tenants) dùng chung một CSDL PostgreSQL 16 nhưng được cách ly tuyệt đối:

2.1. Nhận diện Tenant (`ICurrentTenantService`)

- Mọi request gửi lên API phải đính kèm Header: `Authorization: Bearer <JWT_Token>` .
- Middleware giải mã JWT Token, trích xuất Claim `tenantId` .
- Giá trị `tenantId` này được gán vào `CurrentTenantService` có vòng đời Scoped (tồn tại trong suốt một HTTP Request).

2.2. Lớp lọc tự động (Global Query Filter)

Trong `AppDbContext.OnModelCreating` , 18 thực thể nghiệp vụ đều được áp dụng bộ lọc toàn cục:

```
modelBuilder.Entity<Product>()
  .HasQueryFilter(e => _currentTenantService.TenantId != null && e.TenantId ==
(_currentTenantService.TenantId ?? Guid.Empty));
```

- Bất kỳ câu lệnh LINQ / SQL SELECT nào được sinh ra bởi EF Core đều tự động bị nối thêm mệnh đề `WHERE tenant_id = @currentTenantId` .
- Lập trình viên không cần viết `.Where(x => x.TenantId == ...)` thủ công ở từng service, loại bỏ 100% rủi ro quên lọc dẫn đến rò rỉ dữ liệu giữa các Shop.

2.3. Lớp chốt chặn ghi (Tenant Guard)

Trong `AppDbContext.SaveChangesAsync`:

1. **Kiểm tra tính bất biến (Immutability):** Nếu bản ghi đang bị sửa (`Modified`) và thuộc tính `TenantId` bị thay đổi so với giá trị ban đầu trong DB → Ném ngoại lệ `SecurityException: TenantId is immutable and cannot be changed`.
2. **Kiểm tra quyền sở hữu:** Nếu bản ghi bị sửa hoặc xóa mà `TenantId != currentTenantId` → Ném ngoại lệ `SecurityException`.
3. **Kiểm tra liên kết chéo (Navigation Cross-tenant):** Quét toàn bộ thực thể liên kết, nếu phát hiện thực thể trỏ đến khóa ngoại của Tenant khác → Chặn đứng với lỗi `Cross-tenant reference violation`.

2.4. Kênh đặc quyền kiểm soát (`OpenPrivilegedScope`)

- Khi người dùng gọi API Đăng ký (`RegisterTenant`) hoặc Đăng nhập (`Login`), tại thời điểm đó Client chưa có Token và chưa có `CurrentTenantId`.
- Phương thức `_context.OpenPrivilegedScope()` trả về một `IDisposable` kích hoạt cờ `IsPrivilegedScopeActive = true` cho phép tạo Tenant và User mới hợp lệ, sau đó đóng phạm vi để trả lại trạng thái bảo vệ nghiêm ngặt.

3. ĐẶC TẢ CHI TIẾT 10 CONTROLLERS & 46 ENDPOINTS API

3.1. AuthController (5 Endpoints) Nhựt Phùng Minh phụ trách chính

Quản lý vòng đời xác thực, đăng ký cửa hàng và cấp phát Token (`/api/auth`).

1. POST /api/auth/register-tenant

- **Quyền hạn:** Công khai (Anonymous).
- **Mục đích:** Khởi tạo đồng thời Cửa hàng (Tenant) mới và Tài khoản Chủ shop (Owner) đầu tiên trong một giao dịch ACID.
- **Request Body:** `{ tenantName, tenantCode, fullName, email, password, phoneNumber }`.
- **Logic nghiệp vụ:**
  1. Kiểm tra tính hợp lệ qua `RegisterTenantValidator` (Mật khẩu ≥ 8 ký tự, đủ độ phức tạp hoa/thường/số/ký tự đặc biệt).
  2. Chuẩn hóa Email về chữ thường (`ToLowerInvariant()`), chuẩn hóa SĐT qua `PhoneHelper` về chuẩn quốc tế E.164, chuẩn hóa `TenantCode` về chữ in hoa.
  3. Dùng `.IgnoreQueryFilters()` quét toàn bộ CSDL kiểm tra Email và SĐT đã tồn tại chưa → Nếu có trả về `409 Conflict`.
  4. Mở `OpenPrivilegedScope()` và Database Transaction: Tạo Tenant (`IsActive = true`), băm mật khẩu bằng thuật toán **BCrypt** (`BCrypt.Net.BCrypt.HashPassword(password)`), tạo User gán `TenantId = tenant.Id` , vai trò `Role = "Owner"` . Commit transaction an toàn.
- **Response:** `201 Created` kèm `tenantId, userId, tenantCode` .

2. POST /api/auth/login

- **Quyền hạn:** Công khai (Anonymous).
- **Mục đích:** Xác thực danh tính người dùng bằng Email/Password, cấp cặp JWT Access Token và Refresh Token.
- **Request Body:** `{ email, password }` .
- **Logic nghiệp vụ:**
  1. Dùng `.IgnoreQueryFilters().Include(u => u.Tenant)` tìm User theo Email trên toàn hệ thống.
  2. Nếu không tìm thấy → Trả về `401 Unauthorized` ("Email hoặc mật khẩu không chính xác", chống tấn công dò quét tài khoản Username Enumeration).
  3. So khớp mật khẩu qua `BCrypt.Verify(password, user.PasswordHash)` → Sai trả về `401 Unauthorized` .
  4. Kiểm tra trạng thái: Nếu `!user.IsActive` → Báo tài khoản bị khóa; nếu `!user.Tenant.IsActive` → Báo cửa hàng tạm ngưng.

5. Cấp phát **JWT Access Token** (thời hạn 60 phút, ký bằng thuật toán HMAC-SHA256, chứa Claims: `userId, tenantId, tenantCode, email, role`).
6. Sinh **Refresh Token ngẫu nhiên** (thời hạn 7 ngày). Lưu mã băm SHA-256 (`token_hash`) vào bảng `refresh_tokens`.

- **Response:** `200 OK` kèm `accessToken, refreshToken, expiresIn = 3600, user profile`.

3. POST /api/auth/refresh-token

- **Quyền hạn:** Công khai (Anonymous).
- **Mục đích:** Làm mới phiên làm việc khi Access Token hết hạn thông qua cơ chế bảo mật **Token Rotation**.
- **Logic nghiệp vụ:** Băm token gửi lên bằng SHA-256, tìm kiếm trong bảng `refresh_tokens`. Nếu token không tồn tại, đã hết hạn (`ExpiresAt < UtcNow`) hoặc đã bị thu hồi (`RevokedAt != null`) → Trả về `401 Unauthorized`. Thu hồi token cũ: Gán `existingToken.RevokedAt = DateTimeOffset.UtcNow`. Cấp phát một cặp Access Token và Refresh Token hoàn toàn mới, lưu băm mới vào DB và liên kết `existingToken.ReplacedByToken = newRefreshToken`.
- **Response:** `200 OK` kèm cặp Token mới.

4. POST /api/auth/logout

- **Mục đích:** Đăng xuất an toàn bằng cách thu hồi Refresh Token. Đánh dấu `RevokedAt = DateTimeOffset.UtcNow` vô hiệu hóa vĩnh viễn phiên làm việc.
- **Response:** `200 OK` ("Đăng xuất thành công").

5. GET /api/auth/me

- **Quyền hạn:** `[Authorize]`.
- **Mục đích:** Lấy thông tin cá nhân và quyền hạn người dùng đang đăng nhập dựa trên JWT Claims (`id, tenantId, tenantCode, tenantName, email, fullName, roles`).

3.2. BranchController (5 Endpoints) Lê Võ Đăng Khoa phụ trách

Quản lý các Chi nhánh cửa hàng của Tenant (`/api/branches`).

- `GET /api/branches` : Lấy danh sách tất cả chi nhánh thuộc Tenant hiện tại. Phân quyền: Mọi vai trò đã đăng nhập (`Owner, Staff, Cashier`).
- `GET /api/branches/{id}` : Lấy thông tin chi tiết 1 chi nhánh theo ID. Trả về `404 Not Found` nếu ID không thuộc Tenant.
- `POST /api/branches` : Tạo chi nhánh mới. Phân quyền: `[Authorize(Roles = "Owner")]` . Kiểm tra tên chi nhánh không được rỗng, tạo bản ghi với `IsActive = true` .
- `PUT /api/branches/{id}` : Cập nhật thông tin chi nhánh (Tên, địa chỉ, số điện thoại). Phân quyền: `[Authorize(Roles = "Owner")]` .
- `PUT /api/branches/{id}/toggle-status` : Bật/Tắt trạng thái hoạt động của chi nhánh. Phân quyền: `[Authorize(Roles = "Owner")]` . Chặn không cho xóa cứng chi nhánh để bảo tồn lịch sử giao dịch.

3.3. WarehousesController (5 Endpoints) Khoa & Nhựt phối hợp

Quản lý các Kho hàng vật lý trực thuộc chi nhánh (`/api/warehouses`).

- `GET /api/warehouses` : Lấy danh sách kho hàng. Hỗ trợ lọc theo `branchId` . Mọi vai trò đều được xem.
- `GET /api/warehouses/{id}` : Lấy chi tiết kho hàng kèm thông tin chi nhánh quản lý.
- `POST /api/warehouses` : Tạo kho hàng mới. Phân quyền: `[Authorize(Roles = "Owner,Staff")]` . Kiểm tra bắt buộc `branchId` phải tồn tại và thuộc cùng Tenant.
- `PUT /api/warehouses/{id}` : Cập nhật tên kho, địa chỉ. **Kiểm tra logic BE2-08 (Nhựt đóng góp):** Nếu kho đã có phát sinh tồn kho hoặc giao dịch, nghiêm cấm đổi sang `branchId` khác để tránh sai lệch số dư kho 2 cấp.
- `PUT /api/warehouses/{id}/toggle-status` : Đổi trạng thái hoạt động của kho (`IsActive`).

3.4. SuppliersController (3 Endpoints) Nhựt Phùng Minh phụ trách chính

Quản lý Nhà cung cấp hàng hóa (`/api/suppliers`).

- `GET /api/suppliers` : Lấy danh sách nhà cung cấp của cửa hàng, hỗ trợ tìm kiếm theo tên, SĐT, email.
- `GET /api/suppliers/{id}` : Xem chi tiết nhà cung cấp và lịch sử các đợt nhập hàng.
- `POST /api/suppliers` : Tạo nhà cung cấp mới. Phân quyền: `[Authorize(Roles = "Owner,Staff")]` . Kiểm tra trùng lặp mã số thuế/SĐT trong cùng Tenant.

3.5. CategoriesController (4 Endpoints) Nhựt Phùng Minh phụ trách chính

Quản lý Danh mục sản phẩm phân cấp cây cha - con (`/api/categories`).

- `GET /api/categories`: Trả về toàn bộ danh mục dưới dạng cây phân cấp (Tree Hierarchy). Tải toàn bộ danh mục lên RAM, lọc các nút gốc (`ParentId == null`) và đệ quy lồng ghép các danh mục con vào mảng `Children`.
- `POST /api/categories`: Tạo danh mục mới. Phân quyền: `[Authorize(Roles = "Owner,Staff")]`. Nếu chọn `ParentId`, kiểm tra danh mục cha phải tồn tại và cùng Tenant.
- `PUT /api/categories/{id}`: Cập nhật danh mục. Phân quyền: `[Authorize(Roles = "Owner,Staff")]`.
  - **Cơ chế khóa bảo vệ:** Lấy PostgreSQL Advisory Lock `SELECT pg_advisory_xact_lock(...)` theo `tenantId || '/category_tree'` để chống race condition.
  - **Chống vòng lặp đệ quy (Thuật toán 4.4):** Kiểm tra `ParentId != id` và duyệt tổ tiên ngược dòng. Nếu danh mục cha mới lại là con/cháu của danh mục hiện tại → Trả về `400 Bad Request`.
- `DELETE /api/categories/{id}`: Xóa danh mục. Phân quyền: `[Authorize(Roles = "Owner")]`. Kiểm tra an toàn: Nếu danh mục đang chứa sản phẩm hoặc có danh mục con → Chặn xóa và trả về `400 Bad Request`.

3.6. BrandsController (4 Endpoints) Nhựt Phùng Minh phụ trách chính

Quản lý Thương hiệu sản phẩm (`/api/brands`).

- `GET /api/brands`: Lấy danh sách thương hiệu của cửa hàng.
- `POST /api/brands`: Tạo thương hiệu mới. Phân quyền: `[Authorize(Roles = "Owner,Staff")]`. Chặn trùng tên trong cùng Tenant.
- `PUT /api/brands/{id}`: Cập nhật tên, mô tả thương hiệu. Phân quyền: `[Authorize(Roles = "Owner,Staff")]`.
- `DELETE /api/brands/{id}`: Xóa thương hiệu. Phân quyền: `[Authorize(Roles = "Owner")]`. Chặn xóa nếu đã có sản phẩm liên kết.

3.7. ProductsController (10 Endpoints) Nhựt Phùng Minh phụ trách chính

Quản lý Sản phẩm đại diện và các Biến thể SKU lưu kho (`/api/products`).

- `GET /api/products` : Tra cứu danh sách sản phẩm có phân trang, hỗ trợ tìm kiếm toàn văn theo Tên SP, Mã SKU, Mã vạch Barcode, lọc theo Danh mục và Thương hiệu.
- `GET /api/products/{id}` : Lấy chi tiết sản phẩm kèm toàn bộ danh sách biến thể SKU, giá bán lẻ, giá bán sỉ và thuộc tính JSONB.
- `POST /api/products` : Tạo sản phẩm mới kèm danh sách biến thể SKU trong 1 Transaction nguyên tử. Phân quyền: `[Authorize(Roles = "Owner,Staff")]` .
  - Chống trùng SKU trong mảng gửi lên và trong database của Tenant.
  - Chống trùng Barcode trong mảng gửi lên và trong database của Tenant.
  - Chống trùng tổ hợp thuộc tính biến thể thông qua chữ ký **CombinationSignature** (Thuật toán 4.6).
  - Tự động sinh mã vạch chuẩn quốc tế **EAN-13** nếu người dùng để trống (Thuật toán 4.5).
- `POST /api/products/{id}/variants` : Thêm biến thể SKU mới vào sản phẩm đã có.
- `PUT /api/products/{productId}/variants/{variantId}` : Sửa thông tin biến thể (mã SKU, barcode, thuộc tính động JSONB).
- `PUT /api/products/{productId}/variants/{variantId}/prices` : Cập nhật bảng giá biến thể (Giá bán lẻ `RetailPrice` , Giá bán sỉ `WholesalePrice`).
- `PUT /api/products/{productId}/variants/{variantId}/toggle-status` : Bật/Tắt trạng thái hoạt động của biến thể SKU.
- `DELETE /api/products/{productId}/variants/{variantId}` : Xóa biến thể SKU (chỉ cho phép khi SKU chưa từng phát sinh giao dịch nhập/xuất kho).
- `PUT /api/products/{id}` : Cập nhật thông tin chung của sản phẩm (Tên, mô tả, danh mục, thương hiệu).
- `DELETE /api/products/{id}` : Vô hiệu hóa sản phẩm (`IsActive = false`). Phân quyền: `[Authorize(Roles = "Owner")]` .

3.8. GoodsReceiptsController (4 Endpoints) Lê Võ Đăng Khoa phụ trách chính

Lập chứng từ và Duyệt nhập kho hàng hóa từ nhà cung cấp (`/api/goods-receipts`).

- `POST /api/goods-receipts` : Lập phiếu nhập kho ở trạng thái Nháp (`Draft`). Phân quyền: `[Authorize(Roles = "Owner,Staff")]` . Cho phép thêm nhiều dòng sản phẩm, chỉ định Kho nhập, Nhà cung cấp, Số lượng và Đơn giá nhập của từng dòng.
- `GET /api/goods-receipts` : Tra cứu danh sách phiếu nhập kho, lọc theo trạng thái (`Draft, Confirmed`), thời gian, kho nhập.
- `GET /api/goods-receipts/{id}` : Xem chi tiết phiếu nhập kho, danh sách mặt hàng, số tiền, người tạo, người duyệt.

- `POST /api/goods-receipts/{id}/confirm`: Duyệt xác nhận phiếu nhập kho nguyên tử (Thuật toán 4.1). Phân quyền: `[Authorize(Roles = "Owner,Staff")]`. Chuyển trạng thái phiếu từ `Draft` → `Confirmed`. Thực thi toàn bộ chuỗi: Cộng tồn kho 2 cấp → Tính Giá vốn WAC → Ghi Sổ cái bất biến trong cùng 1 Database Transaction.

3.9. InventoryController (4 Endpoints) Lê Võ Đăng Khoa phụ trách chính

Tra cứu Tồn kho 2 cấp, Lịch sử Sổ cái và Giữ chỗ kho (`/api/inventory`).

- `GET /api/inventory/balances`: Tra cứu số dư tồn kho. Mọi vai trò (`Owner, Staff, Cashier`) đều được gọi. Hỗ trợ lọc theo `branchId, warehouseId, searchTerm`. Trả về chi tiết: Tồn thực tế (`OnHand`), Tồn giữ chỗ (`Reserved`), và Tồn khả dụng thực tế (`Available = OnHand - Reserved`).
- `GET /api/inventory/balances/sku/{skuId}/branch/{branchId}`: Tra cứu nhanh số dư tồn kho của 1 SKU cụ thể tại 1 Chi nhánh.
- `GET /api/inventory/ledger`: Truy vấn Lịch sử Sổ cái kho bất biến (`inventory_transactions`). Phân quyền: `[Authorize(Roles = "Owner,Staff")]` (Thu ngân `Cashier` bị chặn 403 Forbidden vì đây là dữ liệu tài chính nhạy cảm). Trả về: Loại giao dịch (`IN/OUT`), số lượng biến động, số dư tức thời sau biến động (`BalanceAfter`), mã chứng từ và số thứ tự `SequenceNo`.
- `POST /api/inventory/reserve`: Giữ chỗ kho nguyên tử phục vụ bán hàng đa kênh. Phân quyền: `Owner, Staff, Cashier`. Kiểm tra tồn khả dụng: `Available ≥ RequestedQty`. Cơ chế Idempotency với `RequestKey` UUID và PostgreSQL Advisory Lock chống race condition giữ chỗ lặp.

3.10. PosCatalogController (2 Endpoints) Nhựt Phùng Minh phụ trách chính

Tối ưu hóa tìm kiếm và quét mã vạch cho máy POS bán lẻ tại quầy (`/api/pos`).

- `GET /api/pos/search`: Tìm kiếm sản phẩm siêu tốc cho màn hình thu ngân POS. Hỗ trợ tìm theo tên hoặc SKU, trả về danh sách SKU kèm giá bán niêm yết và số lượng tồn khả dụng (`Available`) tại đúng chi nhánh thu ngân đang đứng.
- `GET /api/pos/barcode/{barcode}`: Quét mã vạch Barcode. Tìm chính xác SKU tương ứng với độ trễ cực thấp (< 50ms) để tự động bắn vào giỏ hàng POS.

4. CÁC THUẬT TOÁN & LOGIC NGHIỆP VỤ CỐT LÕI

4.1. Thuật toán Duyệt nhập kho & Tính WAC trong Database Transaction

Lê Võ Đăng Khoa cài đặt

Được cài đặt trong `ConfirmReceiptCommand.cs`:

```
await using var transaction = await _context.Database.BeginTransactionAsync(ct);
try {
  // 1. Kiểm tra trạng thái phiếu nhập
  if (receipt.Status != GoodsReceiptStatus.Draft)
    throw new InvalidOperationException("Phiếu nhập đã được xác nhận trước đó.");

  foreach (var item in receipt.Items) {
    // 2. Cập nhật tồn thực tế kho vật lý
    var whBalance = await GetOrCreateWarehouseBalance(receipt.WarehouseId, item.SkuId);
    whBalance.OnHand += item.Quantity;

    // 3. Cập nhật tồn thực tế và tính WAC tại Chi nhánh
    var brBalance = await GetOrCreateBranchBalance(receipt.BranchId, item.SkuId);
    decimal oldStock = brBalance.OnHand;
    decimal oldCost = brBalance.AverageCost;
    decimal importQty = item.Quantity;
    decimal importPrice = item.UnitPrice;

    // Công thức WAC chuẩn:
    decimal newWac = (oldStock <= 0)
      ? importPrice
      : ((oldStock * oldCost) + (importQty * importPrice)) / (oldStock + importQty);

    brBalance.OnHand += importQty;
    brBalance.AverageCost = Math.Round(newWac, 2);

    // 4. Ghi bút toán vào Sổ cái bất biến
    _context.InventoryTransactions.Add(new InventoryTransaction {
      TenantId = receipt.TenantId,
      WarehouseId = receipt.WarehouseId,
      BranchId = receipt.BranchId,
      SkuId = item.SkuId,
      Type = "IN",
      Quantity = item.Quantity,
      BalanceAfter = brBalance.OnHand,
      ReferenceId = receipt.Id,
      SequenceNo = ++lastSeq
    });
  }

  receipt.Status = GoodsReceiptStatus.Confirmed;
  await _context.SaveChangesAsync(ct);
  await transaction.CommitAsync(ct);
} catch {
  await transaction.RollbackAsync(ct);
  throw;
}
```

4.2. Cơ chế Sổ cái kho bất biến (Append-Only Inventory Ledger) Lê Võ Đăng Khoa cài đặt

Cài đặt Trigger trên PostgreSQL trong `oisms_schema.sql`:

```
CREATE OR REPLACE FUNCTION public.oisms_deny_mutation() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'Operation % on % is prohibited', TG_OP, TG_TABLE_NAME USING ERRCODE='2
END;
$$;

CREATE TRIGGER trg_deny_inventory_transactions_mutation
BEFORE UPDATE OR DELETE ON public.inventory_transactions
FOR EACH ROW EXECUTE FUNCTION public.oisms_deny_mutation();
```

Bất kỳ câu lệnh UPDATE hay DELETE nào tác động lên bảng `inventory_transactions` (kể cả từ code C# hay gõ SQL trực tiếp) đều bị Database ngắt và quăng lỗi từ chối ngay lập tức.

4.3. Cơ chế Giữ chỗ chống bán vượt & PostgreSQL Advisory Lock Idempotency (F2)

Lê Võ Đăng Khoa cài đặt

Được cài đặt trong `ReservationService.cs`:

```
// 1. Chiếm giữ Advisory Lock phân tán theo cặp (tenantId, requestKey)
var lockKey = $"{tenantId}/request_key/{dto.RequestKey}";
await _context.Database.ExecuteSqlInterpolatedAsync(
  $"SELECT pg_advisory_xact_lock(hashtextextended({lockKey}, 0))", ct);

// 2. Đối soát Idempotent
var existingReservation = await _context.Reservations
  .FirstOrDefaultAsync(r => r.TenantId == tenantId && r.RequestKey == dto.RequestKey, ct)

if (existingReservation != null) {
  if (existingReservation.SkuId == dto.SkuId && existingReservation.Quantity == dto.Quant
    // Cùng RequestKey và cùng payload -> Trả về kết quả cũ, KHÔNG tăng Reserved
    return new ReserveStockResult { Success = true, ReservationId = existingReservation.I
  }
  // Cùng RequestKey nhưng khác danh sách hàng -> Báo lỗi xung đột
  return new ReserveStockResult { Success = false, ErrorMessage = "RequestKey trùng lặp v
}

// 3. Kiểm tra Tồn khả dụng thực tế
var balance = await _context.WarehouseInventoryBalances
  .FirstOrDefaultAsync(w => w.WarehouseId == dto.WarehouseId && w.SkuId == dto.SkuId, ct)
var available = balance.OnHand - balance.Reserved;
if (available < dto.Quantity) {
  return new ReserveStockResult { Success = false, ErrorMessage = "Không đủ tồn khả dụng
}

// 4. Tăng Reserved và sinh bản ghi Reservation
balance.Reserved += dto.Quantity;
_context.Reservations.Add(new Reservation {
  TenantId = tenantId,
  RequestKey = dto.RequestKey,
  WarehouseId = dto.WarehouseId,
  SkuId = dto.SkuId,
  Quantity = dto.Quantity,
  ExpiresAt = DateTimeOffset.UtcNow.AddMinutes(15),
  Status = "Active"
});
await _context.SaveChangesAsync(ct);
```

4.4. Thuật toán Chống vòng lặp Cây danh mục (Cycle Detection)

Nhựt Phùng Minh cài đặt

Được cài đặt trong `CategoriesController.cs`:

1. **Khóa cây danh mục:** Lấy `pg_advisory_xact_lock` theo `tenantId || '/category_tree'`.
2. **Duyệt ngược dòng (Ancestor Traversal):**

```
var currentAncestorId = request.ParentId;
while (currentAncestorId.HasValue) {
  if (currentAncestorId.Value == id) {
    // Phát hiện cha mới nằm trong nhánh con cháu của danh mục hiện tại -> Chu trình!
    return BadRequest(new { message = "Không thể chuyển danh mục thành con của chính con
  }
  var parentCategory = await _context.Categories.AsNoTracking()
    .FirstOrDefaultAsync(c => c.Id == currentAncestorId.Value);
  currentAncestorId = parentCategory?.ParentId;
}
```

4.5. Thuật toán Sinh mã vạch chuẩn quốc tế EAN-13 (Modulo 10) Nhựt Phùng Minh cài đặt

Được cài đặt trong `BarcodeHelper.cs`:

1. Sinh chuỗi ngẫu nhiên gồm 12 chữ số: `d1 d2 d3 ... d12`.
2. Tính tổng trọng số chẵn lẻ:

```
Sum = (d1 + d3 + d5 + d7 + d9 + d11) × 1 + (d2 + d4 + d6 + d8 + d10 + d12) × 3
```

3. Tính số kiểm tra thứ 13 (`d13`):

```
d13 = (10 − (Sum mod 10)) mod 10
```

4. Ghép thành mã 13 chữ số hoàn chỉnh và kiểm tra tính duy nhất trong CSDL.

4.6. Cơ chế Thuộc tính động JSONB & Canonical Combination Signature

Nhựt Phùng Minh cài đặt

Được cài đặt trong `SkuAttributeHelper.cs`:

- Bảo toàn 100% cấu trúc JSON lồng nhau và các giá trị `null` bên trong Object / Array.
- Chuẩn hóa chữ ký tổ hợp (**Canonical Signature**) bằng cách sắp xếp theo thứ tự bảng chữ cái của Key và thêm tiền tố kiểu dữ liệu (`s:` cho string, `n:` cho number, `b:` cho boolean).
- **Ví dụ:** `{"size": "L", "color": "Đen"}` và `{"color": "Đen", "size": "L"}` đều được chuẩn hóa thành:

```
o:{color:s:đen,size:s:l}
```

→ Ngăn chặn 100% việc tạo trùng biến thể SKU trong cùng sản phẩm.

5. MA TRẬN PHÂN QUYỀN TRUY CẬP (RBAC MATRIX)

Nhựt Phùng Minh đối soát & hoàn thiện

Nhóm API / Endpoint	Owner (Chủ shop)	Staff (Quản lý kho)	Cashier (Thu ngân POS)	Mã phản hồi vi phạm
Xác thực & Cá nhân (`/api/auth/*`)	Toàn quyền	Toàn quyền	Toàn quyền	`401 Unauthorized`
Cấu hình Chi nhánh (`/api/branches`)	Xem, Tạo, Sửa, Đổi trạng thái	Chỉ xem (GET)	Chỉ xem (GET)	`403 Forbidden`
Cấu hình Kho hàng (`/api/warehouses`)	Toàn quyền	Xem, Tạo, Sửa	Chỉ xem (GET)	`403 Forbidden`
Nhà cung cấp (`/api/suppliers`)	Toàn quyền	Xem, Tạo	Không có quyền	`403 Forbidden`
Danh mục & Thương hiệu (`/api/categories`, `/api/brands`)	Xem, Tạo, Sửa, Xóa	Xem, Tạo, Sửa	Chỉ xem (GET)	`403 Forbidden`
Sản phẩm & SKU (`/api/products`)	Xem, Tạo, Sửa, Xóa	Xem, Tạo, Sửa	Chỉ xem (GET)	`403 Forbidden`
Lập phiếu nhập kho (`POST /api/goods-receipts`)	Toàn quyền	Toàn quyền	Không có quyền	`403 Forbidden`
Duyệt nhập kho & WAC (`POST /api/goods-receipts/{id}/confirm`)	Toàn quyền	Toàn quyền	Không có quyền	`403 Forbidden`
Xem Sổ cái kho Append-Only (`GET /api/inventory/ledger`)	Toàn quyền	Toàn quyền	**Bị chặn (403)**	`403 Forbidden`
Tra cứu Tồn kho 2 cấp (`GET /api/inventory/balances`)	Toàn quyền	Toàn quyền	Toàn quyền	`403 Forbidden`
Giữ chỗ kho bán hàng (`POST /api/inventory/reserve`)	Toàn quyền	Toàn quyền	Toàn quyền	`403 Forbidden`
POS Tra cứu & Quét Barcode (`/api/pos/*`)	Toàn quyền	Toàn quyền	Toàn quyền	`403 Forbidden`

6. PHỤ LỤC: ĐỐI CHIẾU NHIỆM VỤ JIRA CỦA NHỰT PHÙNG MINH

Đối chiếu nghiệm thu

Nhằm phục vụ công tác thanh tra và vấn đáp đồ án với Giảng viên hướng dẫn, toàn bộ kết quả đóng góp của sinh viên **Nhựt Phùng Minh** được liên kết trực tiếp với các mã công việc chính thức trên Jira và Git PR:

Mã Task Jira	Tên Nhiệm vụ	Sản phẩm bàn giao	Trạng thái kiểm chứng
OISMS-16	Nghiên cứu Bối cảnh & Khảo sát Các hệ thống hiện có	Mục I.1, I.2, I.3 trên Confluence và file `docs/CONFLUENCE_I_PROJECT_INTRODUCTION.md`. Ma trận so sánh OISMS với KiotViet, Sapo, Haravan, Odoo.	✓ Đạt Xuất bản tài liệu Confluence trang 983053.
OISMS-27	Bổ sung API Quản lý Kho hàng & Catalog Backend	6 Controllers, 28 Endpoints, Thuật toán EAN-13, Cycle Detection, JSONB Signature, băm mật khẩu BCrypt, Token Rotation.	✓ Đạt PR #4 đã gộp vào `develop`. 194/194 bài test tự động Passed 100%.
OISMS-31	Ma trận phân quyền truy cập màn hình (3.1.3)	Mục 3.1.3 trên Confluence và file `docs/CONFLUENCE_3_1_3_SCREEN_AUTHORIZATION_MATRIX.md` bao phủ 39 màn hình giao diện chuẩn hóa.	✓ Đạt Đã loại bỏ hoàn toàn /forgot-password và các route sai lệch. Đã kiểm chứng trên Admin và POS.

Tài liệu được kết xuất từ mã nguồn thực tế của dự án OISMS — Hoàn tất nghiệm thu Sprint 2.